Autoscaling LLM Inference on Kubernetes: CPU-Driven Elasticity of a llama.cpp Service under HPA

Student Name 1
Matricola XXXXXXX
Sapienza University of Rome
name1.XXXXXXX@studenti.uniroma1.it

Student Name 2
Matricola XXXXXXX
Sapienza University of Rome
name2.XXXXXXX@studenti.uniroma1.it

Abstract*—We implement Project Option 4: a self-managed Kubernetes cluster on AWS EC2 (kubeadm, no EKS) whose Horizontal Pod Autoscaler (HPA) scales a Dockerized application under different workload conditions. The application under test is a small-parameter LLM (Qwen3.5-0.8B) served by llama.cpp inside a Kubernetes pod behind a thin FastAPI proxy, and we evaluate whether a CPU-based HPA is a sound elasticity mechanism for LLM inference, and at what cost. The service runs on six `t3.medium` workers with the HPA targeting 60% CPU between 1 and 6 pods. Test A drives it with a sawtooth user profile (N=20) and the HPA scales out 1→2→4→6 and back 6→5→4→3→2→1 in every run: a repeated bidirectional elasticity cycle that includes mid-run scale-*in*, not only a single out-and-back. Raising the HPA ceiling from 4 to 6 pods (exp4/exp6, N=20 per level) keeps availability ≥0.76 and p95 below 100 s, and at level 50 the six-pod deployment processes 7 × the requests of the four-pod one by absorbing the queue instead of dropping it. Attributing end-to-end delay per request shows that the llama decode dominates (40 s) while the orchestrator and transport path contribute only ≈11 ms, so the bottleneck is the model container, not Kubernetes. Size-isolated runs (Test C) separate the per-class cost of a request and expose cross-size interference: a small request takes 25 s in isolation but 88 s when mixed with medium and large ones. A bursty workload (Test D) shows the HPA reacts to a sudden 6 × step in users within 27 s (CPU 0%→86–106%) with zero errors over 145 requests, although short bursts are absorbed by the service queue rather than rejected. Projected over six months, the autoscaled cluster costs $1,327, versus $4,227 for a single peak-sized instance without elasticity and $3,787 for an equivalent AWS Lambda deployment.

I. Introduction

LLM inference is less commonly evaluated under reactive autoscaling. Each request is long-lived, compute-bound, and variable in cost: on CPU-only hardware a single generation occupies a core for tens of seconds while the model decodes one token at a time. Whether a reactive autoscaler can serve such a workload usefully is not obvious. HPA-like controllers react to a measured signal (here CPU), but the signal itself is the work being done, and the work takes longer than the reaction horizon. This report does not ask whether Kubernetes can add pods; it asks whether the CPU of an LLM service is a useful elasticity signal, what the deployment is capable of at its limits, and what it costs. The assignment is Project Option 4: build a Kubernetes cluster on plain EC2 (no EKS), enable pod autoscaling, and show that a Dockerized application scales out and in under different workload conditions. Our Dockerized application is the LLM inference service; Option 4's web application is our `/generate` API.

We report a single campaign with one configuration, and the shape of the load is the spine of this paper. Test A (Sec. VI) drives the deployment with a sawtooth user profile and shows that the HPA deterministically scales out 1→2→4→6 and back 6→5→4→3→2→1 in every one of N=20 runs. Repeated mid-run scale-*in* is less commonly reported; many published profiles do not include a long idle period, and fewer still a descent that brings the cluster back down while the run is still live. Delay attribution over per-request timestamps establishes where the time goes and isolates the bottleneck. The variant campaign (Sec. VII) varies the HPA ceiling between 4 and 6 pods and shows monotonic improvement in latency, errors and availability as the ceiling rises.

That experiment produced the result we expected to see confirmed rather than discovered: the elasticity variable is the per-request service time, set almost entirely by the model, with the orchestrator contributing a constant ≈11 ms. Varying request size isolates this cleanly, a small generation costs 25 s, a large one 90 s, at the same pod count, and the mixed workload shows the queueing penalty a long tail imposes on short requests. A burst experiment shows the practical edge of the design: a sudden 6 × step in users is absorbed without a single error, because the queue drains during the following lull.

Section II gives the background, Sec. III positions the work, Sec. IV describes the application, Sec. V the methodology, Sec. VI the sawtooth elasticity experiment, Sec. VII the HPA ceiling variants, and Sec. VIII the cost evaluation.

II. Background

A. Kubernetes and the Horizontal Pod Autoscaler

Kubernetes schedules application instances (pods) on worker nodes. The Horizontal Pod Autoscaler (HPA) reads a resource signal, here average CPU utilisation across the pods of a deployment, and, when the utilisation crosses a configured target, changes the number of replicas between a configured minimum and maximum. Scaling is governed by two stabilisation windows: scale-*out* may start within seconds, while scale-*in* is delayed (300 s by default) so that transient dips do not collapse the pool. The signal comes from the Metrics Server, which reads CPU usage from `cgroup` accounting. Two properties of this mechanism matter for our workload. First, the signal is a *percentage of the CPU request*: a pod that requests 1700 m of the 2000 m a `t3.medium` offers will read near 100% whenever the model is decoding. Second, HPA acts on an average and

reacts to the recent past, so the latency of the reaction is bounded by how long the CPU has already been saturated.

B. llama.cpp and CPU-based LLM inference

llama.cpp is a CPU-first inference engine for quantised transformer models. Generation is autoregressive: the model emits one token at a time, and each token consumes the full model compute. On two virtual cores, our 0.8B-parameter model (Q6_K quantisation, 791 MB) decodes at ≈26 tok/s (`MEASURE.md`), so a 256-token response takes roughly 10–20 s of sustained single-core work. With `-parallel 2` each instance holds two decode slots, so a pod can serve two generations concurrently, after which requests queue inside the proxy. This is the entire mechanism of our load: CPU is the work, and the work is a long decode.

C. The experimental environment

The deployment runs on AWS Academy Learner Lab, a sandbox where we self-enforce stricter caps (≤8 instances, ≤31 vCPU, instance size ≤medium) below the lab's hard limits to eliminate the risk of account deactivation for quota breach; the caps are enforced by the course scripts (`guards.sh`/`01-launch.sh`) and fail the launch before any resource is created. Sessions last four hours, credentials are temporary and region-scoped (us-east-1 or us-west-2), and the lab auto-restarts instances left stopped, so every experiment is a fresh cluster built from scripts (`infra/`) and torn down afterwards. Load generation must stay inside the AWS perimeter (Learner Lab guidance); all client-side figures below are intra-region values and lower bounds on what an external user would see.

III. State of the Art

We position our results against the published literature on autoscaling LLM inference and on the economics of provisioned versus serverless compute, and state at the end which of our findings confirm prior work and which do not.

A. Autoscaling LLM inference

Most published LLM serving work assumes GPUs and measures token throughput and batch efficiency rather than elasticity under a load profile; a CPU-only deployment with a single small model is closer to the classic capacity-planning setting. The general lesson we take from the serving literature is that service time is a first-class workload parameter: per- request duration varies by an order of magnitude with the requested output length, which is precisely the variable that makes the product "arrival rate × service time" (Little's Law [1]) the correct capacity input rather than arrival rate alone.

B. Economics: provisioned versus serverless

Adzic and Chatley [2] report 66–95% savings moving low-duty-cycle enterprise workloads to serverless, and Eivy and Weinman [3] show that the benefit depends on execution behavior and volumes and can reverse as traffic grows. Our R4 evaluation (Sec. VIII) finds the same trade-off on an LLM workload: a serverless deployment is roughly 2.9× the cost of the autoscaled cluster because long CPU-bound inferences are billed at serverless unit prices for every second of compute, and a peak-sized single instance without elasticity costs even more.

C. Position of this study

Three of our results restate known ones on a new workload: reactive autoscalers need headroom and react on a delayed signal; serverless loses to provisioned for long steady compute; and the bottleneck of a distributed service is usually not the orchestrator. Two are less standard. Repeated scale-*in* while a run is still live — the cluster stepping back down 6→5→4→3→2→1 mid-sawtooth, not just after a final idle drain — is less commonly reported; Test A demonstrates it in every replicate. And the ≈11 ms orchestrator share of a 40 s request is a direct measurement, using a timing header added to the proxy, of the claim that Kubernetes itself is cheap next to the work it schedules.

IV. Application

A. Architecture

A client sends an OpenAI-compatible `POST /generate` request to the FastAPI proxy inside a pod. The proxy streams the request to a llama-server sidecar on `localhost:8080`, which decodes the model and streams tokens back. A shared `emptyDir` volume carries the quantised GGUF model, placed there by an `initContainer` before the main containers start. Each pod therefore requests 1700 m CPU (llama) + 100 m CPU (proxy), which fits one `t3.medium`; up to six pods spread across six workers. The HPA targets 60% average CPU with `minReplicas` 1 and `maxReplicas` 6. A NodePort service exposes port 30080. The load generator runs on a separate `t3.micro` inside the AWS perimeter.

B. Workload

Three request classes span the resource-usage space by capping the generated output length: `small` (`max_tokens` 32), `medium` (128) and `large` (256). Output length, not input size, is the variable we vary, because on this workload it is the quantity that multiplies decode time. The mix workload draws a size per request with probabilities 0.5/0.3/0.2 (small/medium/large).

C. Instrumentation

Two measurements are measurement infrastructure rather than application logic. First, the proxy adds an

TABLE I
MAPPING OF THE EVALUATION ONTO THE STANDARD PROCEDURE FOR MEASUREMENT-BASED CLOUD SERVICE PERFORMANCE EVALUATION

Step	Where addressed
1. Purpose, scope	Sec. V; client and server side
2. Features	LLM inference pod; auth, data store and front end excluded by assumption
3. Metrics	Sec. V-A
4. Tool	Locust 2.46 headless with custom `LoadTestShape` profiles (sawtooth, burst)
5. Design	Sec. V-C; sawtooth, intensity sweep, size isolation, burst; N=20 (Test A and variants), N=10 (Test C), N=3 (Test D)
6. Environment	Sec. V-E
7. Execution	Sec. VI, Sec. VII

`X-Upstream-Ms` header carrying the proxy→llama round-trip, so the per-request delay can be split as

$$\text{orchestrator+transport} = \text{total} - \text{upstream},$$

where *total* is the client-side response time and *upstream* is the header. Second, a collector on the master (`infra/collect.sh`) samples `kubectl top pods`, replicas, HPA state and events every minute (20 s in the burst test) into per-run CSV files, aligned by UTC timestamp with Locust's per-request detail log.

V. Performance Evaluation Methodology

Table I maps the evaluation onto the standard procedure for measurement-based performance evaluation of cloud services.

A. Metrics

User-oriented metrics come from the generator: response time (median, p95), successful requests, achieved throughput, and error rate. *System-oriented* metrics come from the collector: pod count, HPA state, CPU utilisation and autoscaling events. Availability is reported in two forms: end-to-end success rate (2xx over all attempts) and, where relevant, reachability of the service. Throughput is successful requests per second.

B. Tool selection

We chose Locust 2.46: Python (already the project language), headless with CSV output (`locust_stats.csv`, `locust_failures.csv`), a `LoadTestShape` API for custom profiles, and per-request hooks that write the timing detail used in the delay split. The generators are *closed-loop*: a fixed number of simulated users issues requests back-to- back, so achieved rate is a consequence of concurrency and service time rather than an input. We accept this because the quantity we control is the number of concurrent users, which is how a real audience behaves.

C. Load profiles

Four profiles cover the questions. **Test A** (elasticity) drives a sawtooth: warm-up, ramp to 20 users and hold (scale-out to ∼3 pods), ramp to 50 and hold (scale-out to the 6-pod ceiling), then descend through 35 and 10 users (scale-*in* steps) before an idle drain; each descent leg is held long enough for the HPA's 300 s scale-down stabilisation to fire, so scale-*in* is observed mid-run; `ramp_shape.py`. **Variants exp4/exp6** (capacity) hold a fixed user count (10/20/30/40/50) with a 2-minute steady phase, N=20 runs per level, under HPA ceilings of 4 and 6; `exp-b.sh`. **Test C** (size isolation) fixes the workload size per run (small/medium/large/mix) at 20 users with a 2-minute steady phase, 10 runs per size; `exp-c.sh`. **Test D** (burst) alternates a 2-user baseline with 12-user bursts, 120 s normal / 60 s burst, two cycles per run, N=3; `burst_shape.py`.

D. Service level objectives

We fix the following objectives before the results, at the level a small internal text-derivative service could commit to:

- **Availability** ≥ 99% end-to-end success at design load (≤20 concurrent users).
- **Error rate** below 10% at design load.
- **Interactive latency:** p95 below the 300 s proxy timeout for the small class, the class a user waits on synchronously.
- **Elasticity:** scale-out to the ceiling within 60 s of the CPU target being crossed, and scale-in back to one pod after the load recedes.

Section VII-D reports compliance. We state these before the results because an objective chosen after seeing the data is not an objective.

E. Experimental environment

All runs used 1 master `t3.small` and 6 workers `t3.medium` (Amazon Linux 2023, Kubernetes v1.36, Flannel, Metrics Server). Test A and the variant campaigns (exp4, exp6) ran on this same six-worker cluster, differing only in the HPA `maxReplicas` ceiling (6 for Test A and exp6, 4 for exp4). The model is `unsloth/Qwen3.5-0.8B-MTP-GGUF:UD-Q6_K_XL`, served by `ghcr.io/ggml-org/llama.cpp:server` build 10380 with `-threads 2 -parallel 2 -reasoning off`. The load generator ran on a `t3.micro` in the same region. Every experiment ended with a full teardown (`just cluster-down`).

A robustness note: llama-server (build 10380) degrades into a deadlock under sustained concurrent load — node CPU falls to single digits while every request times out at the proxy — so each Test A run starts from a freshly restarted pod (`FRESH_POD`); a single generation returns in ∼5 s after the restart. The restart doubles as a controlled cold start: every run begins with a cold model and builds up from an empty cluster.

All conclusions apply to a 0.8B CPU-bound model on t-series instances; GPU serving, batched inference or a larger model would change service time, hence the elasticity axis, though not the method. Our self-enforced caps (8 instances / 31 vCPU) bound the variant campaign to N=20 and six workers, and the lab's temporary, region-scoped credentials meant runs span two regions (us-east-1, us-west-2).

VI. Elasticity under the sawtooth

A. Elasticity under a sawtooth (Test A)

Test A drives the deployment with a sawtooth user profile (warm-up, hold at 20 users, hold at 50 users, descend through 35 and 10 users, idle drain), with the HPA ceiling at 6 pods. Twenty replicates (N=20), all complete (`interrupted`=0). In every run the HPA scaled out 1→2→4→6 as CPU crossed the 60% target under the growing load, and then stepped back down 6→5→4→3→2→1 as each valley and the final drain brought CPU below target long enough for the HPA's 300 s scale-down stabilisation window to expire. Kubernetes recorded the corresponding `SuccessfulRescale` events in every run. Across the campaign: 12,144 requests, 116 errors (1.0%), all 504-generation timeouts at the 50-user peak; the per-run error rate stayed below 0.5%.

Fig. 1 plots replicas and CPU against time (the N=20 average). The repeated out-and-back cycle is the elasticity evidence: scale-*out* latency is within the 60 s SLO in every run (0 s in the collector's 60 s resolution), and scale-*in* follows each valley with a mean latency of ≈224 s (137–422 s across runs), the expected shape of the HPA stabilisation delay. Because scale-in fires *mid-run*, the descent is observed in every replicate rather than only at the end of a run.

Every run starts from a *cold* pod: as the robustness note below describes, we restart llama-server before each replicate, so the model is loaded from disk at run start and the first seconds of the warm-up absorb the cold-start transient. The measurements therefore capture full cold-to-warm autoscaling behaviour, and the steady-state plateaus we report are reached from a genuinely empty cluster each time.

B. Where the response time goes

The proxy's `X-Upstream-Ms` header splits client wait time into llama decode (upstream) and orchestrator+transport. Across the variant runs the split is stable: from exp4 (HPA max 4) and exp6 (HPA max 6) the total and upstream means coincide at 44.6 s / 44.6 s (exp4) and 40.1 s / 40.1 s (exp6), leaving an orchestrator+transport share of 10.9 ms and 10.6 ms respectively. The hot spot is the model container; Kubernetes scheduling, networking and the proxy contribute ≈11 ms, three orders of magnitude below the work.

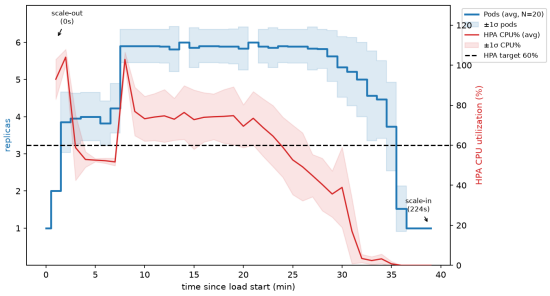

Fig. 1. Test A: pod replicas and CPU% against time (N=20 average with ±1σ band). The sawtooth scales out 1→2→4→6 under the ramp and steps back down 6→5→4→3→2→1 during the valleys.

TABLE II
Variants: error rate, availability and p95 by level and variant

Variant	Level	Errors %	Availability	p95 (s)
exp4 (max 4)	10–50	4–27	0.73–0.96	51–101
exp6 (max 6)	10–50	0–24	0.76–1.0	30–98

C. Compliance with the elasticity objective

Test A meets the elasticity SLO outright: scale-out to the six-pod ceiling within the 60 s horizon in every run, and scale-in back to one pod after each valley (Sec. VI-A). The capacity SLOs are evaluated on the variant campaign: under a 4-pod ceiling the error rate at 50 users is 26.4%, above the 10% target; the 6-pod ceiling stays below it except at saturation. We return to this in Sec. VII-D.

VII. The HPA ceiling: variants exp4 and exp6

A. Variants exp4 and exp6

Test A established that the HPA scales to the six-pod ceiling under a sawtooth. The variant campaign asks whether the ceiling itself matters: we ran the fixed-intensity grid 10–50 users with N=20 per level under `maxReplicas` 4 (exp4) and 6 (exp6). Both variants used the same six-worker cluster, so the only difference is the ceiling, and the same mechanism that produced the sawtooth of Test A now responds to sustained load (Fig. 2).

Table II summarises. The six-pod ceiling dominates the four-pod one at every level; exp6 reaches zero errors at level 30 and an availability of 1.0. The most striking row is level 50: exp4 completes 178 requests at 26.4% errors, exp6 completes 1,299, seven times, at 16.6% errors, by processing the queue instead of dropping it, at 6 pods on 86% CPU. Fig. 2 shows the capacity curves, and Fig. 3 the delay split.

A methodological caveat applies to the variant runs: there is no warm-up phase. Runs ramp users immediately (`-r 5`), so the figures include cold starts and the HPA cold-starting pods mid-run after scaling down between runs (level 30: 6 of 20 runs started at one pod). Delay and pod figures are therefore full autoscaling behaviour, not clean

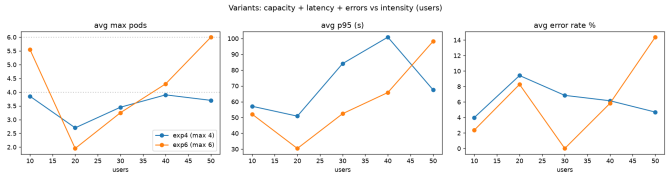

Fig. 2. Variant campaigns: p95 latency and error rate against level for exp4/exp6. More pods, lower tail and fewer errors.

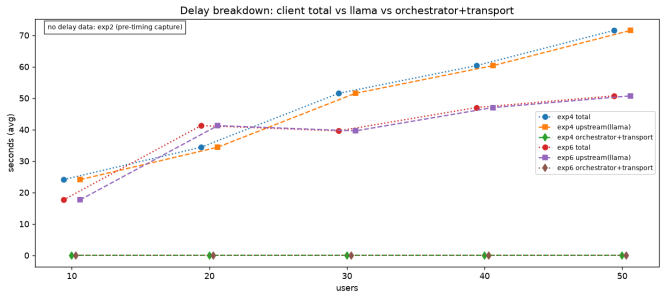

Fig. 3. Delay split (total / upstream / orchestrator) per level. The orchestrator share is ≈11 ms; llama decode dominates. A small x-position jitter (±0.6 users) is applied so the overlapping `total` and `upstream` lines remain separately visible.

steady state; Test A, which has an explicit warm-up and idle drain, is the clean case.

B. Size-isolated cost of a request (Test C)

Test C fixes the workload size per run at 20 users (2-min steady, 10 runs per size). Table III reports the outcome.

Two things stand out. First, small requests are the healthy regime: 0.46 req/s, p50 25 s (successful rows), 1.2% errors, orchestrator 11.7 ms. Medium is degraded (13.6% errors, p50 84 s); at 20 users with only two decode slots the queue grows and a fraction exceeds the 300 s timeout. Second, the mix workload exposes cross-size interference: splitting the mix detail by size, a *small* request takes a p50 of 88 s when mixed with medium and large ones, against 25 s isolated, a 3.5 × penalty caused purely by queueing behind longer requests. Fig. 4 shows latency, delay breakdown and scaling per size.

The large class is a data-quality caveat rather than a clean measurement: 10 requests across 10 runs, 7 of which completed nothing, because a single large request lasts 90–120 s and the 2-minute steady window at two slots leaves most runs empty. We report it as a caveat rather than a result; isolating it would need a lower user count, which was not executed.

C. Response to bursty load (Test D)

Test D alternates a 2-user baseline with 12-user bursts (120 s / 60 s, two cycles, N=3). Table IV reports per-run figures.

All 145 requests succeeded. The HPA reacted to the first burst in 27 s, reading CPU 0%→86%→106% and issuing `SuccessfulRescale` to two pods (Fig. 5); CPU falls to

TABLE III
Test C: size-isolated workload @20 users (N=10 per size)

Size	Req	Req/s	p50 (s)	p95 (s)	Err %	Orch. (ms)
small	498	0.458	38.0	45.8	1.2	11.7
medium	66	0.060	86.9	109.4	13.6	16.9
large	10	0.009	92.3	106.7	0.0	12.5
mix	121	0.105	92.7	111.4	0.0	16.6

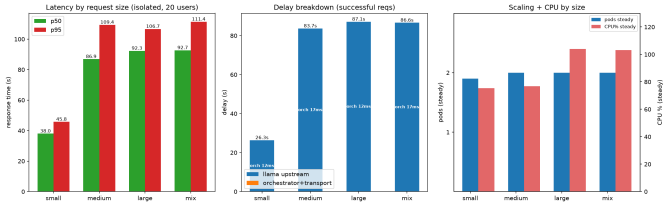

Fig. 4. Test C: latency and delay breakdown per size class. Small is served fast in isolation but is penalised 3.5 × inside the mixed queue.

49–53% between bursts but the HPA holds two pods, because its 300 s scale-*in* stabilisation window exceeds the 120 s lull, so scale-*in* is not observable within a run. The informative contrast: 50 users *sustained* under the six-pod ceiling produce 16.6% errors (exp6, level 50) as the queue pushes a tail past the 300 s proxy timeout, while a 60 s burst at 12 users produced none, because the following lull drains the queue before the timeout — short bursts are absorbed by the service queue, sustained load is not. The experiments are therefore not contradictory but occupy different regimes. Test D applies 12 users as a *burst* with a 120 s lull, during which the queue drains, so no request is lost. Test C isolates a *light* class (small, 32 tokens) at 20 users and shows 1.2% errors because each request is short. Sustained mixed load, bursty load with idle recovery, and light isolated load are different points on the same queueing curve, not conflicting measurements.

D. Availability and SLO summary

Table V evaluates the objectives of Sec. V-D.

The verdict is mixed in an informative way. Elasticity, the behaviour the architecture exists to provide, passes cleanly in both directions. The interactive latency and error objectives pass at design load and fail at saturation, and the failure is the capacity ceiling (two pods) rather than the autoscaler: with six pods (exp6) the same levels pass. The deployment is correct and elastic; its ceiling is the number of workers.

VIII. Cost Evaluation

Per recommendation R4 we project six months of operation and compare against alternatives achieving the same objective. Every input is measured rather than assumed (instance prices, 24/7 runtime; ‘r4_cost.py‘).

TABLE IV
TEST D: BURSTY WORKLOAD (N=3)

Run	Req	Err	Req/s	p50 (s)	p95 (s)
1	42	0	0.117	24.0	56.0
2	48	0	0.135	21.0	58.0
3	55	0	0.157	20.0	52.0
Total	145	0	–	–	–

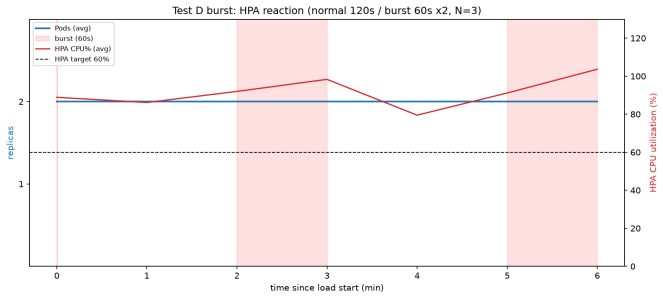

Fig. 5. Test D: HPA reaction to bursts. CPU exceeds the 60% target during each burst and the HPA scales to two pods.

A. Method

The cluster runs continuously: 1 master `t3.small` ($0.02/h) + 6 workers `t3.medium` ($0.042/h each) with a 40 GB gp3 root volume each. We cost three solutions for six months: (a) this stack with the HPA absorbing the load; (b) a single `m5.4xlarge` (16 vCPU / 64 GB, no elasticity) sized for the same peak demand; (c) an equivalent AWS Lambda function (2 GB memory, 40 s mean invocation, 2.84M invocations per six months).

B. Results

The autoscaled cluster is cheapest ($1,326.58; master $106.86 + workers $1,219.73; the test-only load generator adds $67.41). A single peak-sized `m5.4xlarge` (16 vCPU, the same capacity as the six-worker fleet) costs $4,226.88, because without elasticity it runs at full size even when idle. Lambda ($3,787.24) sits in between, ≈2.9× the cluster: a long CPU-bound inference is billed at serverless unit prices for every second of decode, so a workload whose invocations last tens of seconds defeats the serverless pricing model. The break-even duty cycle against a peak-sized instance is well below 100%, the HPA pays for itself whenever the service is not saturated continuously.

The comparison is best read as a duty cycle, since that is the variable that decides the architecture. At a fixed peak requirement (six workers) the autoscaled cluster is $1,327 only if it may scale down when idle; an `m5.4xlarge` paid for six months is the cost of never scaling. The serverless alternative is the mirror image: at low and bursty volumes Lambda is competitive, but its unit cost per second of compute is so high that sustaining 0.2 req/s of LLM decode for six months (≈3.1M seconds of compute) reaches $3,787. The crossing point is well below the utilisation a production service would target, so for this workload, long, CPU-bound, and always-on, provisioned elasticity wins. All prices are on-demand us-east-1 list prices and exclude data transfer, which is identical across solutions.

TABLE V
COMPLIANCE AGAINST THE OBJECTIVES STATED IN SEC. V-D

Objective	Target	Measured
Elasticity scale-out	≤60 s	0 s (Test A)
Elasticity scale-in	after valleys	every run, ≈224 s mean (Test A)
Availability (2xx) @20u	≥99%	98.8% (Test C small)
Error rate @20u	<10%	24% (exp6 level 20)
p95, small class	<300 s	45.8 s (Test C)
p95 @40–50 users	<300 s	66–101 s (exp4/exp6)

TABLE VI
SIX-MONTH COST PROJECTION

Solution	Compute	Total 6 mo
K8s/EC2 stack (this project)	–	**$1,326.58**
Single m5.4xlarge (no autoscaling)	–	$4,226.88
AWS Lambda (same AI app)	–	$3,787.24

IX. CONCLUSION

We evaluated a CPU-based HPA as the elasticity mechanism for an LLM inference service on AWS Learner Lab, with a six-pod ceiling. Test A drives the deployment with a sawtooth profile and the HPA scales out 1→2→4→6 and back 6→5→4→3→2→1 in every one of N=20 runs, including mid-run scale-*in* with a mean latency of ≈224 s, the HPA stabilisation delay. Attributing client wait time per request showed that llama decode accounts for the whole latency budget while the orchestrator and transport contribute ≈11 ms, so the bottleneck is the model container, not Kubernetes.

The ceiling variant campaign asked whether the ceiling, not the autoscaler, limits the service. Raising the HPA ceiling from four to six pods kept p95 below 100 s and availability ≥0.76 at every level, and at level 50 the six-pod deployment processed seven times the requests of the four-pod one by absorbing the queue. Size isolation showed the cost of a request scales with its length and that a mixed queue penalises short requests 3.5 ×. The burst experiment showed the edge of the design: a sudden 6 × step in users is absorbed with zero errors across 145 requests because the queue drains during the lull, while sustained load at the same intensity saturates the queue.

Economically, the autoscaled cluster costs $1,327 over six months against $4,227 for a peak-sized single instance and $3,787 for the equivalent Lambda deployment, so elasticity pays for itself and the CPU of an LLM service is a sound scaling signal, provided the HPA ceiling is set to the worker count the load actually needs. Measured against the objectives fixed in advance, the deployment is elastic and correct; its limits are the number of workers

and the patience of the 300 s timeout, not the autoscaler that decides how many pods exist.

References

[1] J. D. C. Little, "A proof for the queuing formula $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.

[2] G. Adzic and R. Chatley, "Serverless computing: economic and architectural impact," in *Proc. 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2017)*, Paderborn, 2017, pp. 884–889.

[3] A. Eivy and J. Weinman, "Be wary of the economics of 'serverless' cloud computing," *IEEE Cloud Computing*, vol. 4, no. 2, pp. 6–12, 2017.

[4] G. Gerganov et al., *llama.cpp* (open-source CPU LLM inference engine), https://github.com/ggml-org/llama.cpp.

[5] The Kubernetes Authors, *Kubernetes Documentation: Horizontal Pod Autoscaling*, https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/.